ASSESSMENT REMIT

Name			
ID#			
Semester	Fall 2026	**Section**	
School	School of Engineering, Architecture and Interior Design (EAID)		
Program	Bachelor of Science In Computer Science		
Major	Computer Science		
Course Code	**CSCI 310**		
Course Title	Principles of Programming Languages		
Level	☐ Remedial ☐ Bridging ☑ Undergraduate ☐ Graduate		
Assessment Title	Group Assignment – Student Performance Analysis Using Different Programming Paradigms		
Weighting	20 Marks		
Instructor	Dr. Maher Alrahhal		
Hand Out Date	29/10/2026		
Due Date & Time	Date:03/12/2026	Time: 9:00 – 10:59 AM	
Feedback Post Date	10/12/2026		
Assessment Mode	☐ Individual ☑ Group ☐ Closed Book ☑ Open Book		
Assignment Length *(for assignment-based assessments)*	5–8 pages, excluding source code and appendices		
Submission Mode	☐ Online by Email ☑ Online through LMS | TurnItIn ☑ Hard Copy		
Document Type	☐ Words ☑ PDF ☐ Artwork ☑ Software ☐ Other		

GENERAL REQUIREMENTS (GROUP ASSIGNMENT)

- Each group must submit one assignment report with all members' names and Student IDs.
- Complete all four tasks and provide the required Python source code.
- Include program outputs, screenshots and clear explanations where required.
- All submitted programs must be executable and produce correct results.
- Each group member must contribute to the assignment and participate in the demonstration.
- Submit the report and source code through the designated submission platform before the deadline.
- All submitted work must be original. Any plagiarism, unauthorised use of generative AI or academic misconduct will be handled according to University policies.

ASSESSMENT:

Task 1. Programming Language Analysis (5 Marks)

Consider the following Python program:

```
passing_mark = 50

def calculate_result(mark):
    status = "Pass" if mark >= passing_mark else "Fail"
    return status

print(calculate_result(75))
```

a. Identify the global and local variables. (1 mark)

b. Explain how the function accesses passing_mark. (1 mark)

c. Explain the purpose of the conditional expression. (1 mark)

d. Write the exact output of the program. (1 mark)

e. Modify the function so that marks of 85 or above return "Excellent", marks from 50 to 84 return "Pass", and lower marks return "Fail". (1 mark)

Task 2. Solve the Same Problem Using Different Paradigms (9 Marks)

Use the following dataset:

marks = [85, 42, 73, 91, 55, 38, 67]

Develop three implementations to calculate the average mark, count students who passed (mark ≥ 50), and identify the highest mark.

a. Imperative Programming (3 Marks)

Implement the solution using loops, variables and conditional statements. Explain how it processes the data.

b. Functional Programming (3 Marks)

Implement the solution using functions and appropriate functional operations, such as map(), filter() or sum(). Explain how it differs from the imperative approach.

c. Object-Oriented Programming (3 Marks)

Create a StudentPerformance class that stores marks and provides methods to calculate the average, count passing students and find the highest mark. Create an object and demonstrate the results.

Task 3. Evaluate Programming Paradigms (4 Marks)

Compare the three implementations using the following criteria:

- Readability
- Writability
- Code organisation
- Ease of modification

a. Complete a comparison table with brief explanations. (2 marks)

b. Recommend the most suitable paradigm for a university system that may later include attendance, grades and student records. Justify your choice. (1 mark)

c. Explain one limitation of your recommended paradigm. (1 mark)

Task 4. Group Demonstration and Communication (2 Marks)

a. Demonstrate the working programs and explain their results. (1 mark)

b. Ensure all group members participate in explaining the code, comparisons and conclusions. (1 mark)

ANSWER GUIDELINES:

Students are expected to:

- Complete ALL four assignment tasks.
- Provide correct and executable Python programs.
- Clearly explain syntax, semantics, variables, scopes and bindings.
- Implement the student performance problem using imperative, functional and object-oriented approaches.
- Calculate the average mark, number of passing students and highest mark correctly.
- Compare programming paradigms using appropriate evaluation criteria.
- Justify the recommended programming paradigm.
- Demonstrate the working programs and explain the results.
- Ensure every group member contributes to the assignment.

Presentation Requirements

- Include a cover page with all group members' names and Student IDs.
- Organise the report according to Tasks 1–4.
- Include source code, program outputs and brief explanations.
- Provide a comparison table and conclusion.
- Submit the source code files with the report.
- Use APA referencing where external sources are used.
- Clearly identify each member's contribution.

REFERENCING STYLE:

Students should use the American Psychological Association style (APA) as per the following guidelines: https://apastyle.apa.org/style-grammar-guidelines/references/examples

PLAGIARISM & GENERATIVE AI (FOR ASSIGNMENT BASED ASSESSMENTS):

Amity University Dubai is committed to a policy of honesty in all its operations, especially those pertaining to academic matters. It is your responsibility to use appropriate references and apply correct referencing practices. Unless explicitly stated otherwise, the use of generative AI to write an assignment is not permitted. Submission should be yours and should not be in breach of AMUD's Academic Integrity and Academic Misconduct Policy.

LATE SUBMISSIONS (FOR ASSIGNMENT BASED ASSESSMENTS):

You are advised to strictly adhere to the deadline. While Amity University Dubai might consider extenuating circumstances to support students who are experiencing unforeseen issues impacting their ability to complete assessments on time, any anticipated submission delay must be communicated in advance to the instructor who will consult with the Program Leader and the Dean before a permission is granted. All permissions must be based on valid reasons backed up by concrete evidence. For more details, you are advised to consult the latest version of AMUD's Academic Integrity and Academic Misconduct Policy.

FEEDBACK PLAN:

Formative feedback will be provided during regular class sessions and office hours to support students in developing, testing and comparing their programming solutions.
Students will receive guidance on programming language analysis, implementation of different programming paradigms and group collaboration.
Summative written feedback will be provided by **10 December 2026**, based on the submitted report, source code, programming outputs, evaluation and group demonstration.

ASSESSED COURSE LEARNING OUTCOMES:	RELATED QUESTIONS	MAX SCORE	MARK
CLO2: Analyze syntax, semantics, names, scopes and bindings for programming languages.	Task 1	**5**	
CLO3: Solve problems using a range of programming paradigms and communicate the effectiveness of each paradigm for a particular problem, working in teams.	Task 2	**9**	
CLO4: Evaluate and communicate the principles of an imperative, functional, object-oriented, or logic-oriented programming language, working in teams.	Tasks 3–4	**6**	
	TOTAL	**20**	

EMBEDDED PLO DIRECT ASSESSMENT QUESTIONS? ☑ **YES** ☐ **NO**
[IF YES, COMPLETE THE TABLE BELOW]

PLO	EMBEDDED QUESTIONS
PLO1: Analyze a complex computing problem and apply principles of computing and other relevant disciplines to identify solutions.	Task 1 (5 marks) – Programming Language Analysis; Task 2 (9 marks) – Programming Paradigm Implementation.
PLO3: Communicate effectively in a variety of professional contexts.	Task 2 (9 marks) – Explanation of programming solutions; Task 3 (4 marks) – Paradigm Evaluation; Task 4 (2 marks) – Group

	Demonstration and Communication.
PLO5: Function effectively as a member or leader of a team, demonstrating leadership, collaboration and project management in achieving objectives related to the discipline.	Task 2 (9 marks) – Collaborative programming implementation; Task 3 (4 marks) – Group evaluation; Task 4 (2 marks) – Group Demonstration and Communication.

SCORING RUBRIC			LEVEL OF ACHIEVEMENT			
	DESCRIPTOR	**WEIGHT**	**4 [87%-100%]**	**3 [73%-86%]**	**2 [60%-72%]**	**1 [0%-59%]**
1	Programming Language Analysis (Task 1)	5	Accurately analyses variables, scope, conditions and outputs; correct modification.	Mostly correct analysis and code with minor errors.	Partial understanding and incomplete modification.	Incorrect or missing analysis and code.
2	Imperative Programming (Task 2A)	3	Correct implementation and clear explanation of loops and conditions.	Mostly correct implementation with minor errors.	Partially working implementation.	Incorrect or missing implementation.
3	Functional Programming (Task 2B)	3	Correct functional implementation and clear explanation.	Mostly correct implementation and explanation.	Partial implementation or weak explanation.	Incorrect or missing implementation.
4	Object-Oriented Programming (Task 2C)	3	Correct class, methods, object creation and outputs.	Mostly correct class and methods.	Partially functional class or methods.	Incorrect or missing implementation.
5	Programming Paradigm Evaluation (Task 3)	4	Comprehensive comparison, well-supported recommendation and limitation.	Mostly accurate comparison and justification.	Basic comparison with limited justification.	Incorrect or missing evaluation.
6	Group Demonstration and Communication (Task 4)	2	Clear demonstration with meaningful participation by all members.	Good demonstration with minor gaps.	Partial demonstration or uneven participation.	Missing or ineffective demonstration.

Student Declaration

By signing this cover page, you acknowledge the instructions above (general requirements) and accept all responsibilities resulting from breaching them as per the related University policies. I hereby declare that it is my original work and I have duly acknowledged and referenced the source of any quotes, thoughts or ideas incorporated in the work.

Student Signature **Date**

To be filled by faculty

FEEDBACK FROM FACULTY:

Faculty Name and Signature	Date